

MySQL Fundamentals to Advanced

1. Introduction to MySQL

MySQL is a relational database management system (RDBMS) — it stores data in structured **tables** made of rows and columns, and uses SQL (Structured Query Language) to interact with that data.

Real-world example: An e-commerce site stores every product, order, and customer in MySQL tables — so a query like “find all orders placed by customer X” can run instantly instead of searching through files manually.

2. Databases and Tables (DDL)

```
-- Create a database
CREATE DATABASE school;
USE school;

-- Create a table
CREATE TABLE students (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL,
  age INT,
  email VARCHAR(100) UNIQUE
);
```

- PRIMARY KEY — uniquely identifies each row
- AUTO_INCREMENT — auto-generates the next ID
- NOT NULL — column cannot be empty
- UNIQUE — no duplicate values allowed

Altering a table:

```
ALTER TABLE students ADD COLUMN gpa DECIMAL(3,2);
ALTER TABLE students DROP COLUMN gpa;
ALTER TABLE students MODIFY COLUMN name VARCHAR(100);
```

Real-world example: A students table is the digital version of a school’s paper enrollment register — each row is one student’s record.

3. Inserting, Updating, Deleting Data (DML)

```
-- Insert
INSERT INTO students (name, age, email) VALUES ('Ahmad', 20, 'ahmad@mail.com');

-- Update
UPDATE students SET age = 21 WHERE name = 'Ahmad';
```

```
-- Delete
DELETE FROM students WHERE id = 5;
```

Real-world example: When a student updates their email in a university portal, that's a single UPDATE statement running behind the scenes.

4. Querying Data (DQL)

```
SELECT * FROM students;
SELECT name, age FROM students WHERE age > 18;
SELECT * FROM students ORDER BY age DESC;
SELECT * FROM students WHERE name LIKE 'A%';
SELECT * FROM students WHERE age BETWEEN 18 AND 25;
SELECT DISTINCT age FROM students;
```

Clause	Purpose
WHERE	filters rows
ORDER BY	sorts results
LIKE	pattern matching
BETWEEN	range filtering
DISTINCT	removes duplicates

Real-world example: A search bar on a job portal filtering “jobs posted in the last 7 days, sorted by salary” is a SELECT ... WHERE ... ORDER BY query.

5. Aggregate Functions

```
SELECT COUNT(*) FROM students;
SELECT AVG(age) FROM students;
SELECT MAX(age), MIN(age) FROM students;
SELECT SUM(gpa) FROM students;

-- With grouping
SELECT age, COUNT(*) FROM students GROUP BY age;
SELECT age, COUNT(*) FROM students GROUP BY age HAVING COUNT(*) > 2;
```

- GROUP BY groups rows sharing a value
- HAVING filters groups (unlike WHERE, which filters rows)

Real-world example: “Average order value per city” on an e-commerce dashboard is an AVG() with GROUP BY city.

6. Relationships and Foreign Keys

```
CREATE TABLE courses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(100),
  student_id INT,
  FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE
CASCADE
);
```

- FOREIGN KEY links one table to another
- ON DELETE CASCADE — deleting a student automatically deletes their related course records

Real-world example: In a school system, deleting a student should also clean up their course enrollments — cascading prevents orphaned data.

7. JOINS

```
-- INNER JOIN — only matching rows from both tables
SELECT students.name, courses.title
FROM students
INNER JOIN courses ON students.id = courses.student_id;

-- LEFT JOIN — all students, even those with no course
SELECT students.name, courses.title
FROM students
LEFT JOIN courses ON students.id = courses.student_id;

-- RIGHT JOIN — all courses, even unassigned ones
SELECT students.name, courses.title
FROM students
RIGHT JOIN courses ON students.id = courses.student_id;

-- SELF JOIN — comparing rows within the same table
SELECT a.name AS student1, b.name AS student2
FROM students a, students b
WHERE a.age = b.age AND a.id <> b.id;
```

Real-world example: Showing “student name + enrolled course” together on a report card is exactly what an INNER JOIN does — pulling matching data from two related tables.

8. Subqueries

```
SELECT name FROM students
WHERE id IN (SELECT student_id FROM courses WHERE title = 'Math');
```

Real-world example: “Show me all students enrolled in Math” — the inner query finds relevant student IDs first, the outer query fetches their names.

9. Stored Procedures

A stored procedure is a saved, reusable block of SQL logic.

```
DELIMITER //
CREATE PROCEDURE GetStudentsByAge(IN minAge INT)
BEGIN
    SELECT * FROM students WHERE age >= minAge;
END //
DELIMITER ;

CALL GetStudentsByAge(18);
```

Real-world example: A bank might use a stored procedure for “process monthly interest” — run on a schedule, reused every month without rewriting the logic.

10. Indexes

```
CREATE INDEX idx_name ON students(name);
```

Indexes speed up searches on large tables — similar to a book’s index letting you jump to a page instead of reading every page.

Real-world example: A products table with millions of rows needs an index on `product_name` so searches don’t scan the entire table every time.

Notes by Ahmad — CodeNotes